

AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari



Chameleon\



Method	Syntax	Technical Description
Direct Input	<code>user_val = input("Prompt")</code>	Pauses execution to capture a string from the keyboard.
File Reading	<code>open("file.txt", "r").read()</code>	Opens a digital connection to external data stored on the disk.
Hard-coding	<code>data = "Fixed Value"</code>	Directly embedding values into the source code; efficient but rigid.



```
# Moving from Hard-coded to Dynamic: The 'Party Guest' Scenario
```

```
# 1. HARD-CODED (The Boring Way)
```

```
# This script only knows one person. To change it, you have to edit the code.
```

```
guest_name = "Alicia"
```

```
print(f"Welcome to the lab, {guest_name}!")
```

```
# 2. DYNAMIC INPUT (The Interactive Way)
```

```
# The script 'asks' and 'adapts' to whoever is at the keyboard.
```

```
current_user = input("Identify yourself, human: ")
```

```
print(f"Access granted. Hello, {current_user}. Let's analyze some brains.")
```

```
# 3. FILE I/O (The Researcher's Way)
```

```
# in the next slide ...
```


open()



Method / Mode	Syntax	Technical Description
The Context Manager	<code>with open(...) as file:</code>	The gold standard; automatically closes the "pipe" even if the script crashes.
Read Mode ('r')	<code>open("data.txt", "r")</code>	The default setting; opens a file for reading only . Will fail if the file doesn't exist.
Write Mode ('w')	<code>open("data.txt", "w")</code>	Creates a new file or overwrites an existing one. Use with caution!
Append Mode ('a')	<code>open("data.txt", "a")</code>	Adds new data to the end of a file without deleting what's already there.

```
with open('data.txt', 'r') as f:  
    data = f.read()  
# At this point, the file is automatically CLOSED.
```

```
f = open('data.txt', 'r')  
data = f.read()  
f.close()  
# You MUST remember to do this manually
```

open()



```
# The 'Laboratory Notebook' Script: Managing Research Data
```

```
file_name = "experiment_log.txt"
```

```
# 1. WRITING: Creating the log (The 'Nuclear' Option)
```

```
# This will create the file or overwrite it if it exists.
```

```
with open(file_name, "w") as pilot_file:
```

```
    pilot_file.write("Session 01: Participant exhibited high focus.\n")
```

```
    print(f"--- {file_name} initialized. ---")
```

```
# 2. APPENDING: Adding data (The 'Safe' Option)
```

```
# This adds to the end without deleting the previous session.
```

```
new_entry = "Session 02: Participant fell asleep. Send coffee.\n"
```

```
with open(file_name, "a") as log_file:
```

```
    log_file.write(new_entry)
```

```
    print("--- New entry added to the log. ---")
```

```
# 3. READING: Bringing it all back into Python
```

```
# Let's read it as a list so we can count the sessions.
```

```
with open(file_name, "r") as final_report:
```

```
    all_sessions = final_report.readlines()
```

```
print("\n--- Summary Report ---")
```

```
for i, line in enumerate(all_sessions, 1):
```

```
    print(f"Log #{i}: {line.strip()}")
```

```
print(f"\nTotal sessions recorded: {len(all_sessions)}")
```

Keyword Arguments



Argument Type	Calling Syntax	Technical Description
Positional	<code>func("Alpha", 26)</code>	Values are mapped to parameters based strictly on their index/order.
Keyword	<code>func(age=26, name="Alpha")</code>	Values are mapped by name; allows for arbitrary order and high readability.
Mixed	<code>func("Alpha", age=26)</code>	Combines both; positional arguments must come before keyword arguments.

Keyword Arguments



```
def start_match(player_name, character_class, difficulty=5, permadeath=False):  
    print(f"--- Loading Match ---")  
    print(f"Player: {player_name} | Class: {character_class}")  
    print(f"Difficulty Level: {difficulty}")  
    if permadeath:  
        print("WARNING: One life only. Good luck.")  
  
# 1. POSITIONAL: Order is everything.  
# 'X-Wing' is name, 'Pilot' is class.  
start_match("X-Wing", "Pilot")  
  
# 2. KEYWORD: The labels make it clear (and the order can flip!)  
start_match(character_class="Mage", player_name="Zelda", permadeath=True)  
  
# 3. MIXED: The 'Professional' way.  
# Note: player_name (positional) MUST come first.  
start_match("Sonic", character_class="Speedster", difficulty=10)
```




Side Quest!

If using keyword names makes code clearer, why don't we use them for *every single* argument all the time? Is there a downside to being "too descriptive"?

Your reward : 1pt

Default Arguments



Feature	Syntax	Technical Description
Defining Defaults	<code>def func(x, y=10):</code>	Assigns a fallback value to a parameter if the caller provides nothing.
Overriding	<code>func(5, y=20)</code>	Replaces the fallback with a specific user-defined value.
The Rule of Order	<code>(required, default)</code>	Non-default arguments must be listed before default arguments.

```
# The 'Dungeon Crawler' - Setting up a quick adventure
def enter_room(player, loot="Gold Coin", trap_active=True):
    state = "dangerous" if trap_active else "safe"
    print(f"{player} found {loot} in a {state} room.")
```

```
# 1. Lazy: Uses all defaults
enter_room("Hero")
```

```
# 2. Specific: Override one default via keyword
enter_room("Wizard", trap_active=False)
```

*args



Feature	Syntax	Technical Description
The "Packer" (*)	<code>def func(*args):</code>	Collects an arbitrary number of positional arguments into a tuple .
Iteration	<code>for item in args:</code>	Allows the function to perform the same logic on every provided item.
Unpacking	<code>func(*my_list)</code>	Takes an existing list and "spreads" its items into the function as individual arguments.

*args



```
# The 'Party Invite' Packer
def shout_invites(event, *guests):
    print(f"--- Event: {event} ---")
    for guest in guests:
        print(f"Hype Level 100: Welcome {guest}!!")

# 1. Packing: Handing over a random number of items
shout_invites("Coding Night", "Alice", "Bob", "Charlie", "Dave")

# 2. Unpacking: Taking a list and 'spreading' it in
late_arrivals = ["Eve", "Frank"]
shout_invites("After Party", *late_arrivals)
```



Side Quest!

```
def enhance_data(*args):  
    # The Mystery: We want to 'clean' the first item in our collection  
    args[0] = args[0].strip().upper()  
    print(f"Updated collection: {args}")  
  
# The Trigger:  
enhance_data(" low-quality-data ", "high-quality-data")
```

Your reward : 2pt

****kwargs**



Feature	Syntax	Technical Description
The "Dictionary" ()**	<code>def func(**kwargs):</code>	Captures "keyword=value" pairs into a dictionary .
Flexibility	<code>kwargs.get("key")</code>	Allows a function to accept any named setting without pre-defining it.
The Pass-Through	<code>other_func(**kwargs)</code>	Sends a whole bundle of settings into another function (like a plotting tool).



```
# The 'Gaming Engine' - Simple & Flexible
```

```
def render_character(name, **settings):
```

```
    # 1. THE SAFE BET: If no 'outfit' is provided, default to 'Cape'
```

```
    outfit = settings.get("outfit", "Basic Cape")
```

```
    print(f"--- Rendering {name} in a {outfit} ---")
```

```
    # 2. THE HANDOVER: Pass all settings to the 'Physics Engine'
```

```
    apply_physics(**settings)
```

The Master Order



Order	Argument Type	Purpose
1st	Standard (Positional)	Mandatory items that define the core of the task.
2nd	*args	The "infinite suitcase" for extra unlabelled values.
3rd	Defaults	Pre-set settings that serve as fallbacks.
4th	**kwargs	The "infinite filing cabinet" for specific named configurations.

The Master Order



```
# The Master Template: A robust pipeline configuration
def run_simulation(session_id, *data_points, mode="Standard", **metadata):
    print(f"ID: {session_id} | Mode: {mode}")
    print(f"Points captured: {len(data_points)}")
    print(f"Meta: {metadata}")

# Applying the Master Order
run_simulation("SIM_99", 0.5, 0.8, 0.4, location="AMU_Lab", user="Research_Bot")
```



QUIZ!

You have the code; here is the function:

(Previously you changed the code order, now code will change your order)

```
def run_trial(subject_id, *raw_scores, mode="Standard", **metadata):
```

```
    """ Every row is ONE function call. You must sit in order from LEFT to RIGHT. """
```